

FastAPI, Pydantic, SQLite, and Sessions Notes

REST routes, request models, status codes, chat sessions, SQL, persistence, and API testing.

Engineering notes and worked references

Contents

1. REST basics
2. FastAPI routes
3. Pydantic request models
4. Status codes
5. The /chat endpoint
6. SQLite tables
7. Parameterized SQL
8. Transactions and commits
9. Persistence and restart
10. Error handling and tests

1. REST basics

A REST-style API exposes resources and actions through HTTP methods and paths. The method, path, request body, status code, and response body form a contract with the caller.

For a small Python service, clear routes and predictable JSON matter more than trying to follow every possible REST convention.

Example from use

GET reads a resource. POST creates or submits work. PATCH changes part of a resource. DELETE removes a resource.

Things people mix up

- Using GET for an action that changes stored state.
- Returning 200 for every error.
- Changing response field names between requests.

Quick check

- Can the idea be explained without relying on one memorized definition?
- Can a small example be traced from input to output?
- Can the likely failure point be named when the result is wrong?

The useful habit is to inspect the intermediate state. For API work that may be the request and response. For retrieval it may be the retrieved chunks and scores. For session work it may be the stored messages. Seeing the state usually makes the bug less mysterious.

Request lifecycle

For rest basics, trace one request through validation, application logic, database work, and response creation. FastAPI makes the HTTP layer easy, but the application still needs clear boundaries between route code, business logic, storage, and external model calls.

A short route is easier to test because deterministic pieces can be exercised without starting the whole server. Database repository functions, query rewriters, parsers, and source-formatting helpers can all be tested directly.

Useful route tests

- Valid request.
- Missing required field.
- Unknown resource ID.
- Second session to check isolation.
- Restart when persistence is part of the requirement.

If a route fails, keep the user-safe response short while logging enough server detail to find the cause. Do not return raw tracebacks as normal API output.

2. FastAPI routes

FastAPI maps HTTP routes to Python functions. Decorators such as `@app.get` and `@app.post` declare method and path.

Route functions should stay small enough to read. Put retrieval, database, and model logic into normal helper functions or services so they can be tested separately.

Example from use

POST /chat can validate the request and call the conversation pipeline.

GET /notes can list saved notes for a session.

Things people mix up

- Putting the entire application in one route function.
- Using global mutable state for sessions.
- Catching every exception and returning plain text.

Quick check

- Can the idea be explained without relying on one memorized definition?
- Can a small example be traced from input to output?
- Can the likely failure point be named when the result is wrong?

The useful habit is to inspect the intermediate state. For API work that may be the request and response. For retrieval it may be the retrieved chunks and scores. For session work it may be the stored messages. Seeing the state usually makes the bug less mysterious.

Request lifecycle

For fastapi routes, trace one request through validation, application logic, database work, and response creation. FastAPI makes the HTTP layer easy, but the application still needs clear boundaries between route code, business logic, storage, and external model calls.

A short route is easier to test because deterministic pieces can be exercised without starting the whole server. Database repository functions, query rewriters, parsers, and source-formatting helpers can all be tested directly.

Useful route tests

- Valid request.
- Missing required field.
- Unknown resource ID.
- Second session to check isolation.
- Restart when persistence is part of the requirement.

If a route fails, keep the user-safe response short while logging enough server detail to find the cause. Do not return raw tracebacks as normal API output.

3. Pydantic request models

Pydantic models describe the expected JSON shape. FastAPI validates many errors before endpoint logic runs.

Use required fields for values the endpoint cannot operate without. Use optional fields only when there is a defined default behavior.

Example from use

ChatRequest can require session_id and message.

SaveNoteRequest can require title and content.

Things people mix up

- Making every field optional to avoid validation errors.
- Using a string for a value that should be a list or integer.
- Duplicating validation in several route functions.

Quick check

- Can the idea be explained without relying on one memorized definition?
- Can a small example be traced from input to output?
- Can the likely failure point be named when the result is wrong?

The useful habit is to inspect the intermediate state. For API work that may be the request and response. For retrieval it may be the retrieved chunks and scores. For session work it may be the stored messages. Seeing the state usually makes the bug less mysterious.

Request lifecycle

For pydantic request models, trace one request through validation, application logic, database work, and response creation. FastAPI makes the HTTP layer easy, but the application still needs clear boundaries between route code, business logic, storage, and external model calls.

A short route is easier to test because deterministic pieces can be exercised without starting the whole server. Database repository functions, query rewriters, parsers, and source-formatting helpers can all be tested directly.

Useful route tests

- Valid request.
- Missing required field.
- Unknown resource ID.
- Second session to check isolation.
- Restart when persistence is part of the requirement.

If a route fails, keep the user-safe response short while logging enough server detail to find the cause. Do not return raw tracebacks as normal API output.

4. Status codes

Status codes tell the client what happened. Success, validation failure, missing resource, and server failure should not all look the same.

FastAPI commonly returns 422 for request-model validation. A missing saved note is naturally a 404. A new note can return 201 if the API uses that convention.

Example from use

GET /notes/abc -> 404 when abc does not exist.

POST /chat -> 200 when a normal answer is returned.

Things people mix up

- Returning 500 for an unknown note ID.
- Returning 200 with {'error': 'failed'} for every failure.
- Using 404 for a malformed JSON body.

Quick check

- Can the idea be explained without relying on one memorized definition?
- Can a small example be traced from input to output?
- Can the likely failure point be named when the result is wrong?

The useful habit is to inspect the intermediate state. For API work that may be the request and response. For retrieval it may be the retrieved chunks and scores. For session work it may be the stored messages. Seeing the state usually makes the bug less mysterious.

Request lifecycle

For status codes, trace one request through validation, application logic, database work, and response creation. FastAPI makes the HTTP layer easy, but the application still needs clear boundaries between route code, business logic, storage, and external model calls.

A short route is easier to test because deterministic pieces can be exercised without starting the whole server. Database repository functions, query rewriters, parsers, and source-formatting helpers can all be tested directly.

Useful route tests

- Valid request.
- Missing required field.
- Unknown resource ID.
- Second session to check isolation.
- Restart when persistence is part of the requirement.

If a route fails, keep the user-safe response short while logging enough server detail to find the cause. Do not return raw tracebacks as normal API output.

5. The /chat endpoint

A chat endpoint usually accepts a session identifier and the new message, loads the relevant session state, runs the assistant pipeline, stores the new messages, and returns a structured response.

Keep the request and response models explicit so demo clients and tests know what fields to expect.

Example from use

Request: `session_id` and `message`. Response: `answer`, `sources`, optional `retrieved_context`, optional `saved_note_id`.

Store both user and assistant messages after a successful turn.

Things people mix up

- Saving the assistant message before the pipeline succeeds.
- Ignoring `session_id`.
- Returning the full database history when the client only asked for an answer.

Quick check

- Can the idea be explained without relying on one memorized definition?
- Can a small example be traced from input to output?
- Can the likely failure point be named when the result is wrong?

The useful habit is to inspect the intermediate state. For API work that may be the request and response. For retrieval it may be the retrieved chunks and scores. For session work it may be the stored messages. Seeing the state usually makes the bug less mysterious.

Request lifecycle

For the /chat endpoint, trace one request through validation, application logic, database work, and response creation. FastAPI makes the HTTP layer easy, but the application still needs clear boundaries between route code, business logic, storage, and external model calls.

A short route is easier to test because deterministic pieces can be exercised without starting the whole server. Database repository functions, query rewriters, parsers, and source-formatting helpers can all be tested directly.

Useful route tests

- Valid request.
- Missing required field.
- Unknown resource ID.
- Second session to check isolation.
- Restart when persistence is part of the requirement.

If a route fails, keep the user-safe response short while logging enough server detail to find the cause. Do not return raw tracebacks as normal API output.

6. SQLite tables

SQLite is enough for sessions, messages, and saved notes in a small project. The schema should match the data the application actually needs.

Use primary keys and foreign keys where they make the relationships clearer. Add timestamps when ordering matters.

Example from use

sessions(session_id, created_at)

messages(id, session_id, role, content, created_at)

saved_notes(note_id, session_id, title, content, source_refs, created_at)

Things people mix up

- Creating many empty tables because a production app might need them someday.
- Storing all session data as one giant JSON string when queries are needed.
- Using note title as the only identifier.

Quick check

- Can the idea be explained without relying on one memorized definition?
- Can a small example be traced from input to output?
- Can the likely failure point be named when the result is wrong?

The useful habit is to inspect the intermediate state. For API work that may be the request and response. For retrieval it may be the retrieved chunks and scores. For session work it may be the stored messages. Seeing the state usually makes the bug less mysterious.

Request lifecycle

For sqlite tables, trace one request through validation, application logic, database work, and response creation. FastAPI makes the HTTP layer easy, but the application still needs clear boundaries between route code, business logic, storage, and external model calls.

A short route is easier to test because deterministic pieces can be exercised without starting the whole server. Database repository functions, query rewriters, parsers, and source-formatting helpers can all be tested directly.

Useful route tests

- Valid request.
- Missing required field.
- Unknown resource ID.
- Second session to check isolation.
- Restart when persistence is part of the requirement.

If a route fails, keep the user-safe response short while logging enough server detail to find the cause. Do not return raw tracebacks as normal API output.

7. Parameterized SQL

SQL values should be passed as parameters rather than concatenated into a string. This keeps quoting correct and avoids SQL injection problems.

The SQL structure can remain in code while user-supplied values are bound separately.

Example from use

```
cursor.execute('SELECT * FROM messages WHERE session_id = ?', (session_id,))
```

Things people mix up

- Building WHERE clauses with raw user text.
- Putting table names directly from user input.
- Assuming SQLite means injection does not matter.

Quick check

- Can the idea be explained without relying on one memorized definition?
- Can a small example be traced from input to output?
- Can the likely failure point be named when the result is wrong?

The useful habit is to inspect the intermediate state. For API work that may be the request and response. For retrieval it may be the retrieved chunks and scores. For session work it may be the stored messages. Seeing the state usually makes the bug less mysterious.

Request lifecycle

For parameterized sql, trace one request through validation, application logic, database work, and response creation. FastAPI makes the HTTP layer easy, but the application still needs clear boundaries between route code, business logic, storage, and external model calls.

A short route is easier to test because deterministic pieces can be exercised without starting the whole server. Database repository functions, query rewriters, parsers, and source-formatting helpers can all be tested directly.

Useful route tests

- Valid request.
- Missing required field.
- Unknown resource ID.
- Second session to check isolation.
- Restart when persistence is part of the requirement.

If a route fails, keep the user-safe response short while logging enough server detail to find the cause. Do not return raw tracebacks as normal API output.

8. Transactions and commits

Database writes need a clear commit boundary. A note should not appear saved to the user if the transaction later rolls back.

For a small app, a short transaction per request is usually easier to reason about than keeping one connection and transaction open for a long time.

Example from use

Insert a saved note, commit, then return `note_id`.

Rollback when an exception occurs before the write is complete.

Things people mix up

- Returning success before commit.
- Sharing one cursor across unrelated requests.
- Leaving transactions open accidentally.

Quick check

- Can the idea be explained without relying on one memorized definition?
- Can a small example be traced from input to output?
- Can the likely failure point be named when the result is wrong?

The useful habit is to inspect the intermediate state. For API work that may be the request and response. For retrieval it may be the retrieved chunks and scores. For session work it may be the stored messages. Seeing the state usually makes the bug less mysterious.

Request lifecycle

For transactions and commits, trace one request through validation, application logic, database work, and response creation. FastAPI makes the HTTP layer easy, but the application still needs clear boundaries between route code, business logic, storage, and external model calls.

A short route is easier to test because deterministic pieces can be exercised without starting the whole server. Database repository functions, query rewriters, parsers, and source-formatting helpers can all be tested directly.

Useful route tests

- Valid request.
- Missing required field.
- Unknown resource ID.
- Second session to check isolation.
- Restart when persistence is part of the requirement.

If a route fails, keep the user-safe response short while logging enough server detail to find the cause. Do not return raw tracebacks as normal API output.

9. Persistence and restart

A saved note or message is only persistent if it survives a process restart. This is worth testing directly.

Use a known database path and print or log the path during development so you know which file the API is using.

Example from use

Save a note, stop uvicorn, start it again, then GET the note.

Create two sessions and confirm both still exist after restart.

Things people mix up

- Using `'memory:'` while claiming durable storage.
- Writing the database to a temporary directory unknowingly.
- Deleting the database in startup code.

Quick check

- Can the idea be explained without relying on one memorized definition?
- Can a small example be traced from input to output?
- Can the likely failure point be named when the result is wrong?

The useful habit is to inspect the intermediate state. For API work that may be the request and response. For retrieval it may be the retrieved chunks and scores. For session work it may be the stored messages. Seeing the state usually makes the bug less mysterious.

Request lifecycle

For persistence and restart, trace one request through validation, application logic, database work, and response creation. FastAPI makes the HTTP layer easy, but the application still needs clear boundaries between route code, business logic, storage, and external model calls.

A short route is easier to test because deterministic pieces can be exercised without starting the whole server. Database repository functions, query rewriters, parsers, and source-formatting helpers can all be tested directly.

Useful route tests

- Valid request.
- Missing required field.
- Unknown resource ID.
- Second session to check isolation.
- Restart when persistence is part of the requirement.

If a route fails, keep the user-safe response short while logging enough server detail to find the cause. Do not return raw tracebacks as normal API output.

10. Error handling and tests

API tests should cover valid requests, missing fields, bad IDs, session isolation, provider failure, and database behavior.

Separate deterministic code from external model calls so tests can mock the model or retrieval layer where useful.

Example from use

Use FastAPI TestClient for route-level tests.

Mock a provider timeout and check that the API returns a controlled response.

Things people mix up

- Only testing one successful /chat request.
- Calling the real model in every unit test.
- Ignoring the response body when the status code is correct.

Quick check

- Can the idea be explained without relying on one memorized definition?
- Can a small example be traced from input to output?
- Can the likely failure point be named when the result is wrong?

The useful habit is to inspect the intermediate state. For API work that may be the request and response. For retrieval it may be the retrieved chunks and scores. For session work it may be the stored messages. Seeing the state usually makes the bug less mysterious.

Request lifecycle

For error handling and tests, trace one request through validation, application logic, database work, and response creation. FastAPI makes the HTTP layer easy, but the application still needs clear boundaries between route code, business logic, storage, and external model calls.

A short route is easier to test because deterministic pieces can be exercised without starting the whole server. Database repository functions, query rewriters, parsers, and source-formatting helpers can all be tested directly.

Useful route tests

- Valid request.
- Missing required field.
- Unknown resource ID.
- Second session to check isolation.
- Restart when persistence is part of the requirement.

If a route fails, keep the user-safe response short while logging enough server detail to find the cause. Do not return raw tracebacks as normal API output.